In [1]:
```
# EDA: Explatory Data Analysis
```

In [2]:
```
import pandas as pd
```

In [3]:
```
emp = pd.read_excel(r'C:\FSDS_Adv_Gene_AI_&Agentic_AI\Python\EDA_June\Rawdata.xlsx')
```

In [4]:
```
emp
```

Out[4]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [5]:
```
emp.columns
```

Out[5]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

In [6]:
```
emp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       4 non-null      object
 3   Location  4 non-null      object
 4   Salary    6 non-null      object
 5   Exp       5 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

In [7]:
```
emp.head()
```

Out[7]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year

In [8]:
```
emp.tail()
```

Out[8]:

	Name	Domain	Age	Location	Salary	Exp
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [9]:
```
emp.isnull()
```

Out[9]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

In [10]:
```
emp.isna()
```

Out[10]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

In [11]:
```
id(emp)
```

Out[11]:
```
2227330692304
```

In [12]:
```
emp.describe()
```

Out[12]:

	Name	Domain	Age	Location	Salary	Exp
count	6	6	4	4	6	5
unique	6	6	4	4	6	5
top	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
freq	1	1	1	1	1	1

Cleaning the data of Categorial coloumn

In [13]:
```
emp['Name'] = emp['Name'].str.replace(r'\W', '', regex = True)  # Here regular exp/ Special c
emp['Name']
```

Out[13]:
```
0     Mike
1    Teddy
2     Umar
3     Jane
4    Uttam
5      Kim
Name: Name, dtype: object
```

In [14]:
```
emp['Domain'] = emp['Domain'].str.replace(r'\W', '', regex = True)
emp['Domain']
```

Out[14]:
```
0    Datascience
1        Testing
2    Dataanalyst
3      Analytics
4     Statistics
5            NLP
Name: Domain, dtype: object
```

In [15]:
```
emp['Location'] = emp['Location'].str.replace(r'\W', '' , regex = True)
emp['Location']
```

Out[15]:
```
0       Mumbai
1    Bangalore
2          NaN
3     Hyderbad
4          NaN
5        Delhi
Name: Location, dtype: object
```

In [16]:
```
emp['Age'] = emp['Age'].str.extract(r'(\d+)')  # d+ is digit match digit only.
emp['Age']
```

Out[16]:
```
0     34
1     45
2    NaN
3    NaN
4     67
5     55
Name: Age, dtype: object
```

In [17]:
```
emp
```

Out[17]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5^00#0	2+
1	Teddy	Testing	45	Bangalore	10%%000	<3
2	Umar	Dataanalyst	NaN	NaN	1$5%000	4> yrs
3	Jane	Analytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67	NaN	30000-	5+ year
5	Kim	NLP	55	Delhi	6000^$0	10+

In [18]:
```
# Cleaning of Numerical Data
```

In [19]:
```
emp['Salary'] = emp['Salary'].str.replace(r'\W', '', regex = True)
emp['Salary']
```

Out[19]:
```
0     5000
1    10000
2    15000
3    20000
4    30000
5    60000
Name: Salary, dtype: object
```

In [20]:
```
emp['Exp'] = emp['Exp'].str.extract(r'(\d+)')  # d+ is digit match digit only.
emp['Exp']
```

Out[20]:
```
0      2
1      3
2      4
3    NaN
4      5
5     10
Name: Exp, dtype: object
```

In [21]:
```
emp
```

Out[21]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

Missing value treatment

In [22]:
```
import numpy as np
```

In [23]:
```
clean_data = emp.copy()    # Here created copy of clean data
clean_data
```

Out[23]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [24]:
```
clean_data.isna().sum()
```

Out[24]:
```
Name        0
Domain      0
Age         2
Location    2
Salary      0
Exp         1
dtype: int64
```

In [25]:
```
clean_data['Age'] = clean_data['Age'].fillna(np.mean(pd.to_numeric(clean_data['Age'])))
clean_data['Age']
```

Out[25]:
```
0       34
1       45
2    50.25
3    50.25
4       67
5       55
Name: Age, dtype: object
```

In [26]:
```
clean_data['Location'] = clean_data['Location'].fillna(clean_data['Location'].mode()[0])
clean_data['Location']
```

Out[26]:
```
0       Mumbai
1    Bangalore
2    Bangalore
3     Hyderbad
4    Bangalore
5        Delhi
Name: Location, dtype: object
```

In [27]:
```
clean_data['Exp'] = clean_data['Exp'].fillna(np.mean(pd.to_numeric(clean_data['Exp'])))
clean_data['Exp']
```

Out[27]:
```
0      2
1      3
2      4
3    4.8
4      5
5     10
Name: Exp, dtype: object
```

In [28]:
```
clean_data
```

Out[28]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50.25	Bangalore	15000	4
3	Jane	Analytics	50.25	Hyderbad	20000	4.8
4	Uttam	Statistics	67	Bangalore	30000	5
5	Kim	NLP	55	Delhi	60000	10

Now changing Datatypes as per coloumn names.

To Do So we use ASTYPE: Convert system inbuilt data type to user define datatype.

In [29]:
```
type(clean_data)
```

Out[29]:
```
pandas.core.frame.DataFrame
```

In [30]:
```
id(clean_data)
```

Out[30]:
```
2227251212400
```

In [31]:
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       6 non-null      object
 3   Location  6 non-null      object
 4   Salary    6 non-null      object
 5   Exp       6 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

In [32]:
```
clean_data.columns
```

Out[32]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

In [33]:
```
clean_data['Age'] = clean_data['Age'].astype(int)
clean_data['Exp'] = clean_data['Exp'].astype(int)
clean_data['Salary'] = clean_data['Salary'].astype(int)


clean_data['Name'] = clean_data['Name'].astype('category')
clean_data['Domain'] = clean_data['Domain'].astype('category')
clean_data['Location'] = clean_data['Location'].astype('category')
```

In [34]:
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype   
---  ------    --------------  -----   
 0   Name      6 non-null      category
 1   Domain    6 non-null      category
 2   Age       6 non-null      int64   
 3   Location  6 non-null      category
 4   Salary    6 non-null      int64   
 5   Exp       6 non-null      int64   
dtypes: category(3), int64(3)
memory usage: 938.0 bytes
```

In [35]:
```
clean_data
```

Out[35]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Bangalore	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	Bangalore	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [36]:
```
clean_data.to_csv('clean_data.csv')
```

In [37]:
```
import os
os.getcwd()
```

Out[37]:
```
'C:\\Users\\user'
```

In [38]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [39]:
```
import warning
warning.filterwarnings
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[39], line 1
----> 1 import warning
      2 warning.filterwarnings

ModuleNotFoundError: No module named 'warning'
```

In [ ]:
```
clean_data
```

Working on univariate technique(using on 1 variable: Salary Column)

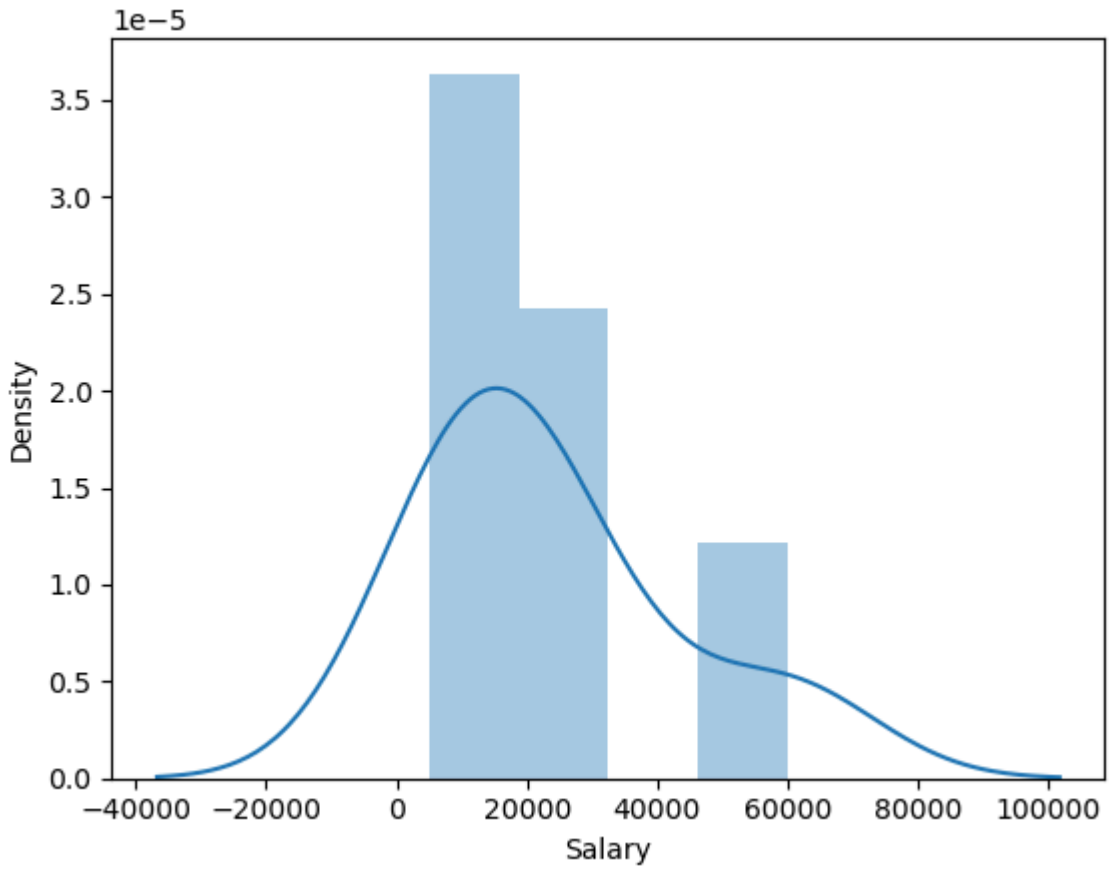
In [40]:
```
clean_data['Salary']
```

Out[40]:
```
0     5000
1    10000
2    15000
3    20000
4    30000
5    60000
Name: Salary, dtype: int64
```

In [41]:
```
vis1 = sns.distplot(clean_data['Salary'])
vis1.show()
```

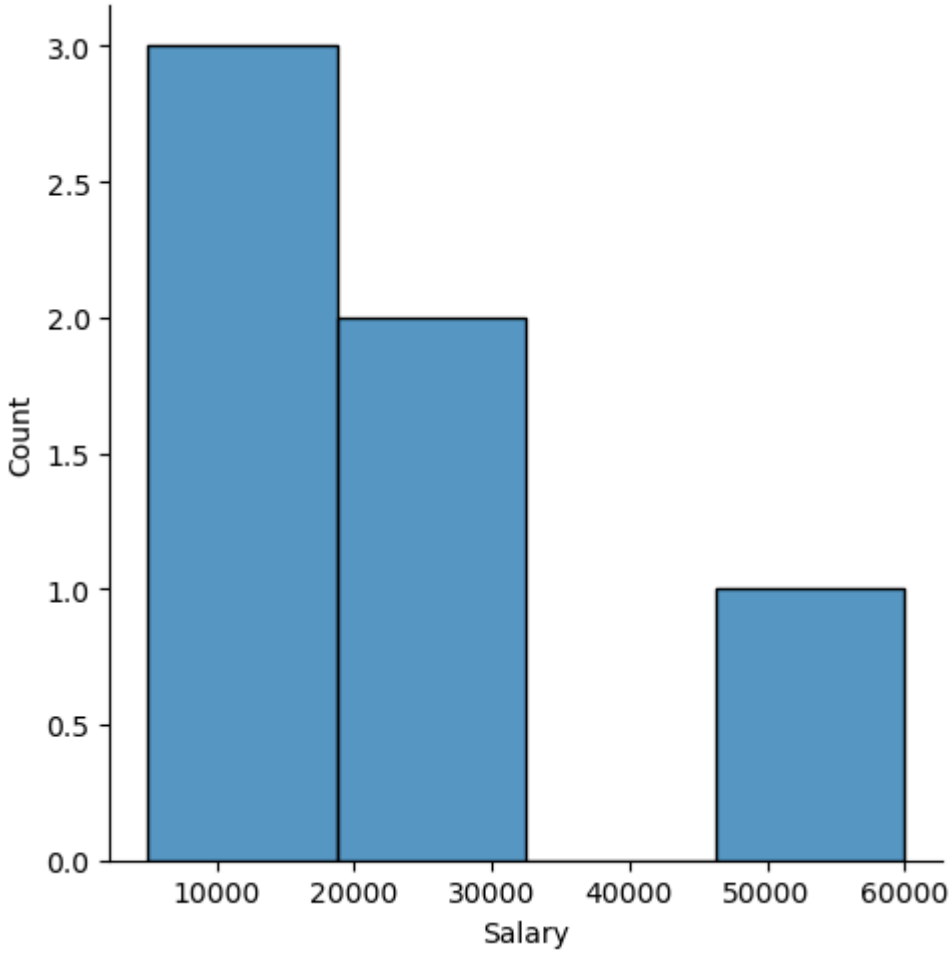
```
C:\Users\user\AppData\Local\Temp\ipykernel_2588\3065846161.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  vis1 = sns.distplot(clean_data['Salary'])
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[41], line 2
      1 vis1 = sns.distplot(clean_data['Salary'])
----> 2 vis1.show()

AttributeError: 'Axes' object has no attribute 'show'
```



In [42]:
```
vis1 = sns.displot(clean_data['Salary'])
vis1.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[42], line 2
      1 vis1 = sns.displot(clean_data['Salary'])
----> 2 vis1.show()

AttributeError: 'FacetGrid' object has no attribute 'show'
```

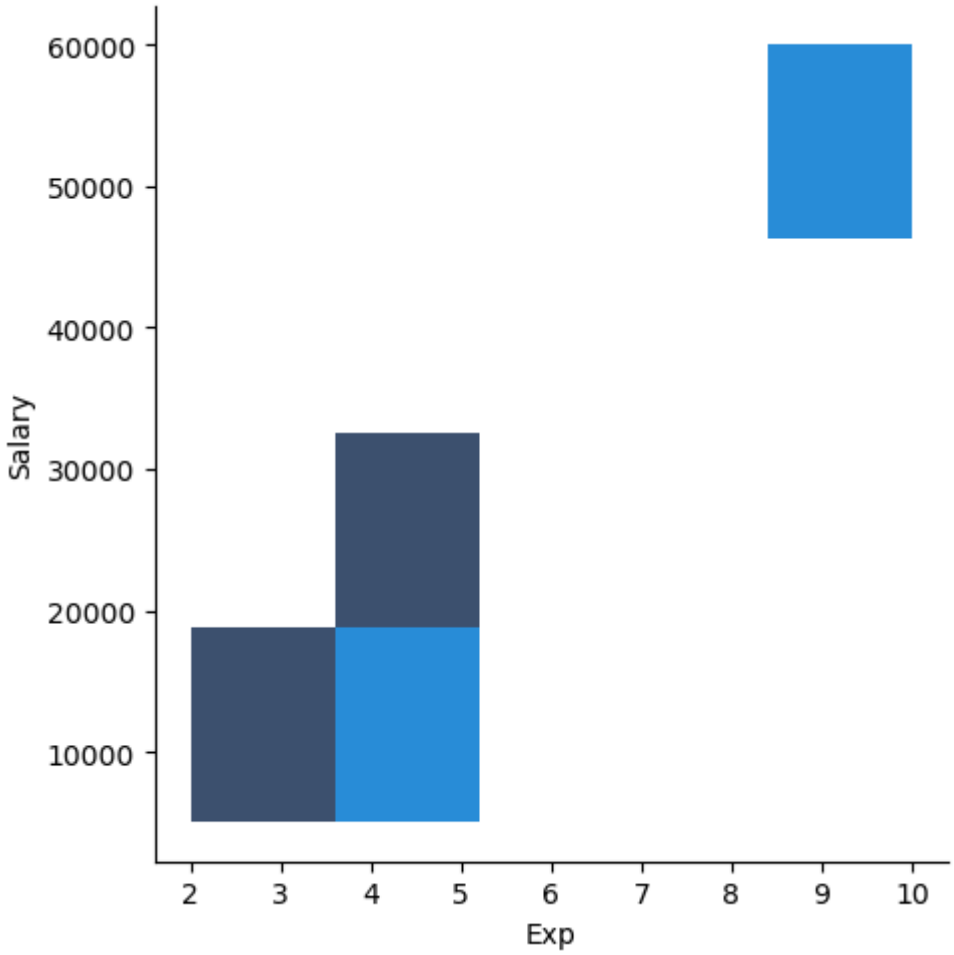


Here in above graph 50000-60000 is outlier

Here using bivariate Technique (using 2 values)

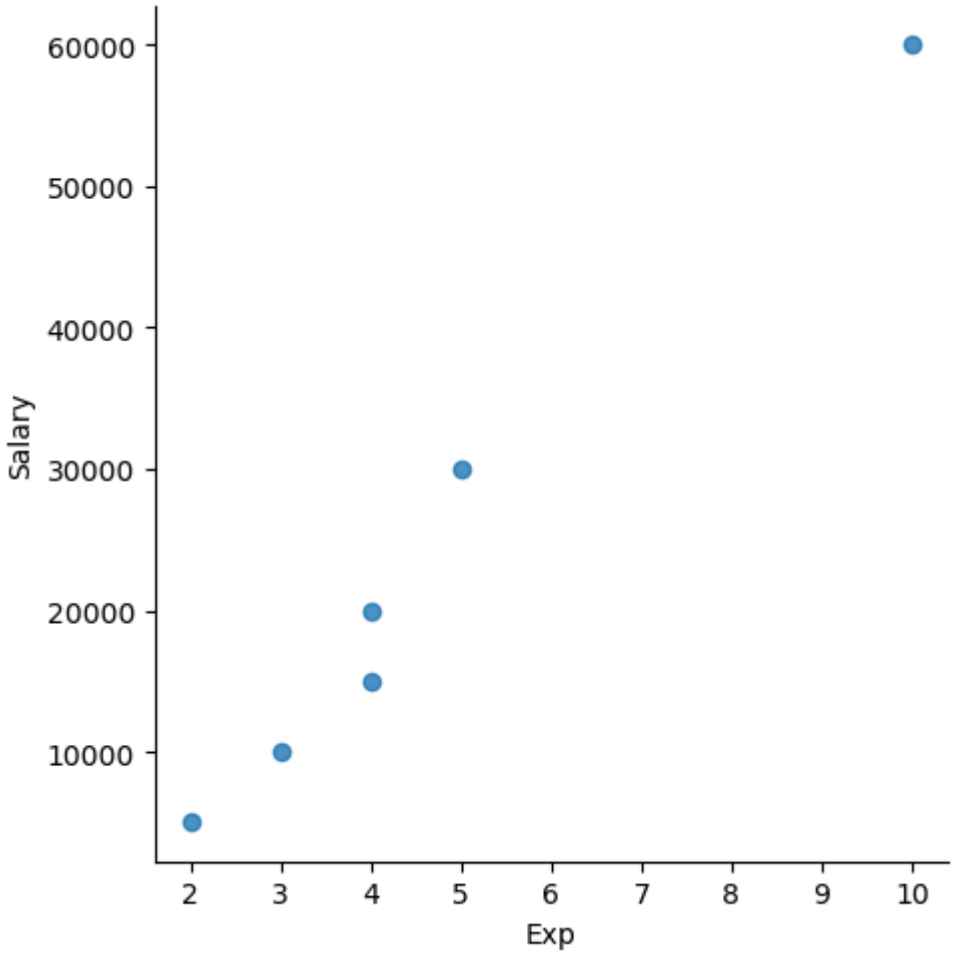
In [43]:
```
vis4 = sns.displot(data = clean_data, x = 'Exp', y = 'Salary' )
vis4.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[43], line 2
      1 vis4 = sns.displot(data = clean_data, x = 'Exp', y = 'Salary' )
----> 2 vis4.show()

AttributeError: 'FacetGrid' object has no attribute 'show'
```



In [44]:
```
vis4 = sns.lmplot(data = clean_data, x = 'Exp', y = 'Salary', fit_reg = False )
vis4.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[44], line 2
      1 vis4 = sns.lmplot(data = clean_data, x = 'Exp', y = 'Salary', fit_reg = False )
----> 2 vis4.show()

AttributeError: 'FacetGrid' object has no attribute 'show'
```



using Slicing to check a single row

In [45]:
```
clean_data = [3:4]
```

```
  Cell In[45], line 1
    clean_data = [3:4]
                  ^
SyntaxError: invalid syntax
```

In [46]:
```
clean_data.columns
```

Out[46]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

Variable identification Technique

In [47]:
```
y_dv = clean_data['Salary'] # dv = Dependent variable
```

In [48]:
```
y_dv
```

Out[48]:
```
0     5000
1    10000
2    15000
3    20000
4    30000
5    60000
Name: Salary, dtype: int64
```

In [52]:
```
x_iv = clean_data[['Name', 'Domain', 'Age', 'Location']]  # iv = independent variable
```

In [53]:
```
x_iv
```

Out[53]:

	Name	Domain	Age	Location
0	Mike	Datascience	34	Mumbai
1	Teddy	Testing	45	Bangalore
2	Umar	Dataanalyst	50	Bangalore
3	Jane	Analytics	50	Hyderbad
4	Uttam	Statistics	67	Bangalore
5	Kim	NLP	55	Delhi

In [ ]:
```
# Imputation Technique (Transform)  # Changing text into numbers
```

In [55]:
```
imputation = pd.get_dummies(clean_data)
imputation
```

Out[55]:

	Age	Salary	Exp	Name_Jane	Name_Kim	Name_Mike	Name_Teddy	Name_Umar	Name_Uttam	D
0	34	5000	2	False	False	True	False	False	False	
1	45	10000	3	False	False	False	True	False	False	
2	50	15000	4	False	False	False	False	True	False	
3	50	20000	4	True	False	False	False	False	False	
4	67	30000	5	False	False	False	False	False	True	
5	55	60000	10	False	True	False	False	False	False	

In [59]:
```
imputation = pd.get_dummies(clean_data, dtype = int)  # All all the rowsare converts to 0\1 v
imputation
```

Out[59]:

	Age	Salary	Exp	Name_Jane	Name_Kim	Name_Mike	Name_Teddy	Name_Umar	Name_Uttam	D
0	34	5000	2	0	0	1	0	0	0	
1	45	10000	3	0	0	0	1	0	0	
2	50	15000	4	0	0	0	0	1	0	
3	50	20000	4	1	0	0	0	0	0	
4	67	30000	5	0	0	0	0	0	1	
5	55	60000	10	0	1	0	0	0	0	

In [ ]:
```

```